

ARM PrimeCell

General Purpose Input/Output (PL061)

Integration Manual

ARM PrimeCell General Purpose Input/Output (PL061) Integration Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell General Purpose Input/Output

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	5
4.4	Synthesis	5
4.4.1	Setup Scripts	5
4.4.2	Command Script	6
4.4.3	Constraint Scripts	6
4.5	Integration with AMBA Micropack components	7
4.6	Interrupt Connection	7
4.7	Clocking (PCLK)	8
4.8	Clock Tree synthesis	8
4.9	Resets	8
4.10	Reset Buffering	8
4.11	Building 16, 24, 32 bit GPIO interfaces using the basic 8 bit PL061.	9
4.12	Integration Testing	10
4.13	Production Testing	10
4.13.1	Scan Insertion and ATPG	10
4.14	Functional and Timing Verification	11
5	APPENDIX A	12
5.1	PrimeCell Environment Variables	12

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A00	22 August, 2000	First draft
A01	11 September, 2000	Added section "Building a 16, 24, 32 bit bit GPIO interfaces using the basic 8 bit PL061".
A	12 September, 2000	First Release

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0190	ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual
3	PL061 DDES 0000	ARM PrimeCell General Purpose Input/Output (PL061) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05
7	SCB00-QPRO-0008-A03	ARM PrimeCell Integration Vector Strategy

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
GPIO	General Purpose Input/Output

2 SCOPE

This document describes how the ARM PrimeCell General Purpose Input/Output (GPIO PL061) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell GPIO is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell block, refer to the ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual [Ref.2].

The ARM PrimeCell General Purpose Input/Output (PL061) Design Manual [Ref.3] contains information about the implementation and design of the ARM PrimeCell GPIO block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

The PrimeCell Integration Vector Strategy document [Ref. 7] describes the motivation and use of integration test vectors.

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell GPIO has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell GPIO block.

An example ASIC design flow is described below:

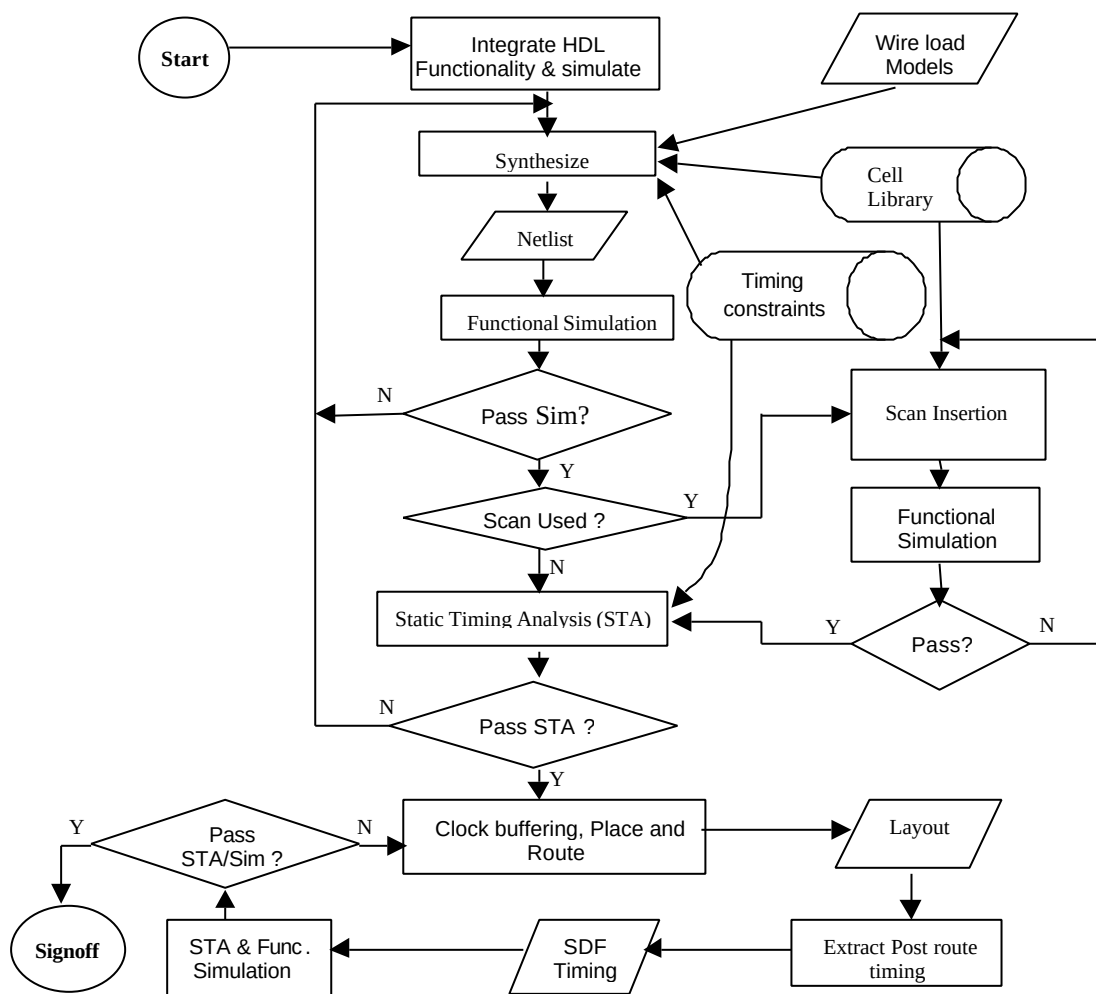


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell GPIO block. For further information refer to the ARM PrimeCell General Purpose Input/Output (PL061) Technical Reference Manual [Ref.2].

Name	Type	Source / Destination	Description
PCLK	In	Clock Generator	APB clock
PRESETn	In	Reset Controller	APB reset
PADDR[11:2]	In	APB Bridge	APB address
PSEL	In	APB Bridge	APB peripheral select
PENABLE	In	APB Bridge	APB enable
PWRITE	In	APB Bridge	APB write signal
PWDATA[7:0]	In	APB Bridge	APB write data
PRDATA[7:0]	Out	APB Bridge	APB read data

Table 4.2-1 APB interface signals

Name	Type	Source / Destination	Description
nGPAFEN[7:0]	In	Generic	Hardware control output enable
GPAFOUT[7:0]	In	Generic	Hardware control data input
GPAFIN[7:0]	Out	Generic	Hardware control data output
GPIOMIS[7:0]	Out	Interrupt Controller	Interrupt signals
GPIINTR	Out	Interrupt Controller	Combined OR version of GPIOMIS
SCANENABLE	In	Test Controller	Scan enable
SCANINPCLK	In	PCLK scan chain	Scan data input signal for PCLK domain
SCANOUTPCLK	Out	PCLK scan chain	Scan data output for PCLK domain

Table 4.2-2: On-Chip Signals

Name	Type	Source / Destination	Description
nGPEN[7:0]	Out	PAD	GPIO Output pad enable signal
GPOUT[7:0]	Out	PAD	GPIO Output data signal driver
GPIN[7:0]	In	PAD	GPIO Input data from pad

Table 4.2-3: Signals to I/O pad cells

4.3 Bus Interconnection

The GPIO block interfaces with the AMBA APB interface using separate uni-directional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the AMBA APB Specification [Ref.1].

The data bus may be interfaced at the APB Bridge either using multiplexers or logical OR'ing of data buses from other APB peripheral blocks. In common with other APB peripheral blocks, the GPIO block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the GPIO block with a 3-state bi-directional data bus. A wrapper needs to be added around the GPIO block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Synthesis

The GPIO block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - GPIO.cmd - synthesizes the GPIO to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the GPIO_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.4.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The library_avanti_cb25_v2.1.scr script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the library_avanti_cb25_v2.1.scr file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The \$LIB_SYNOP directory must contain the files, or links to the target cell library files, in Synopsys format.

4.4.2 Command Script

The master compile command script GPIO.cmd in the synopsys/scripts directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The GPIO.cmd script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys .db format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate log/ directory.

The scan-insertion script GPIO_scan.cmd in the synopsys/scripts directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to 'topname' to define the name of the top level module in the design.
- \$HDL_SOURCE is assigned to 'hdl' to define the input HDL source. HDL_SOURCE should be set to either vhdl or verilog in order to read in the VHDL or Verilog source RTL files.
- \$HDL_NETL is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either vhdl or verilog to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.4.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in the library_avanti_cb25_v2.1.scr file.
- b) Timing constraints for APB bus signals are set via the apb_params.scr and the apb_slave.scr file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The apb_params.scr file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block which is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs applies for a style of APB Bridge design with registered inputs implementing logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA GPIO-specific signals are set via the GPIO.scr file. Sample input and output delay constraints are also specified for the other signals. It should be noted that for asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

4.5 Integration with AMBA Micropack components

The GPIO block is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus.) It is designed to work in systems where an AMBA bus master can read from or write to the GPIO through the APB bridge (from the ASB or AHB) in response to interrupt signals. A dedicated DMA interface is also available to enable connection to external DMA controllers.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the GPIO and any other APB peripherals in the memory map. The base address of the GPIO needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as a slave peripheral on the ASB / AHB with a decode signal, which must be altered in the HDL code of the AMBA decoder block, to select the correct range for the APB peripherals (including the GPIO). If AMBA TIC methodology is to be used for integration testing then any change of address location for the GPIO block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the GPIO integration test program.

4.6 Interrupt Connection

The interrupt logic provides individual outputs, GPIOMIS[7:0], as well as combined interrupt output, GPIOINTR, allowing you to use either a global interrupt service routine or modular device drivers to handle interrupts. A status flag will be able to generate the interrupt request if a corresponding mask flag is set. The interrupt request GPIOINTR is asserted when at least one of the selected status flags is HIGH. The interrupt request block is shown in **Figure 4.6-1**.

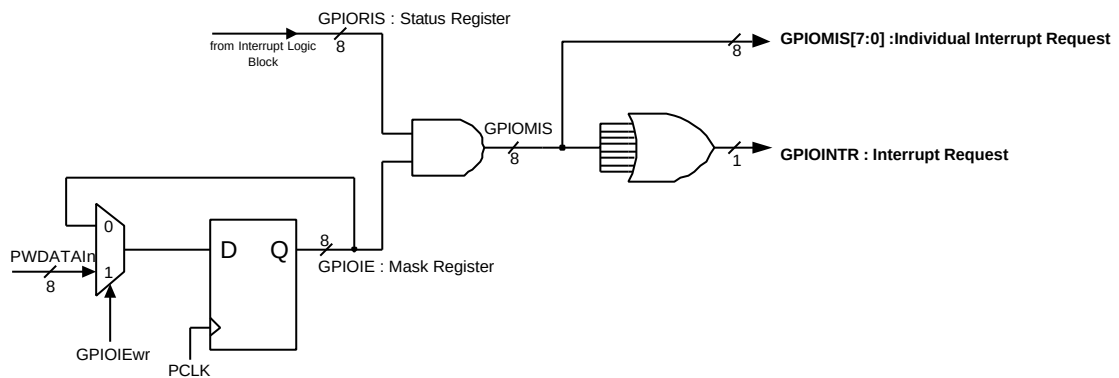


Figure 4.6-1 Interrupt Logic

Note: In Figure 4.6-1, for clarity, test logic is not shown.

There is a separate mask register for each interrupt request signal, GPIOIE.

GPIOINTR would typically be routed to the nIRQ interrupt line of the ARM processor through an external Interrupt Controller.

Note that the GPIOINTR and GPIOMIS[7:0] outputs are asynchronous to PCLK if level detection is selected. When edge detection is selected to trigger interrupt requests GPIOINTR and GPIOMIS are synchronized to PCLK.

4.7 Clocking (PCLK)

The ARM PrimeCell GPIO HDL contains a primary clock source at the top level – PCLK.

It should be noted that the GPIO design assumes that the clock source is free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical applications. It is imperative that the clock is shut off only after the GPIO is completely idle. Accesses to the GPIO are not guaranteed to complete correctly if the clock is not free-running.

4.8 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default GPIO synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.9 Resets

The GPIO block has one reset pin. The system bus reset signal PRESETn, resets each of its internal registers. The reset can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems.

4.10 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

4.11 Building 16, 24, 32 bit GPIO interfaces using the basic 8 bit PL061.

Figure 4.11-1 shows how to create a 24-bit GPIO by instantiating the GPIO three times.

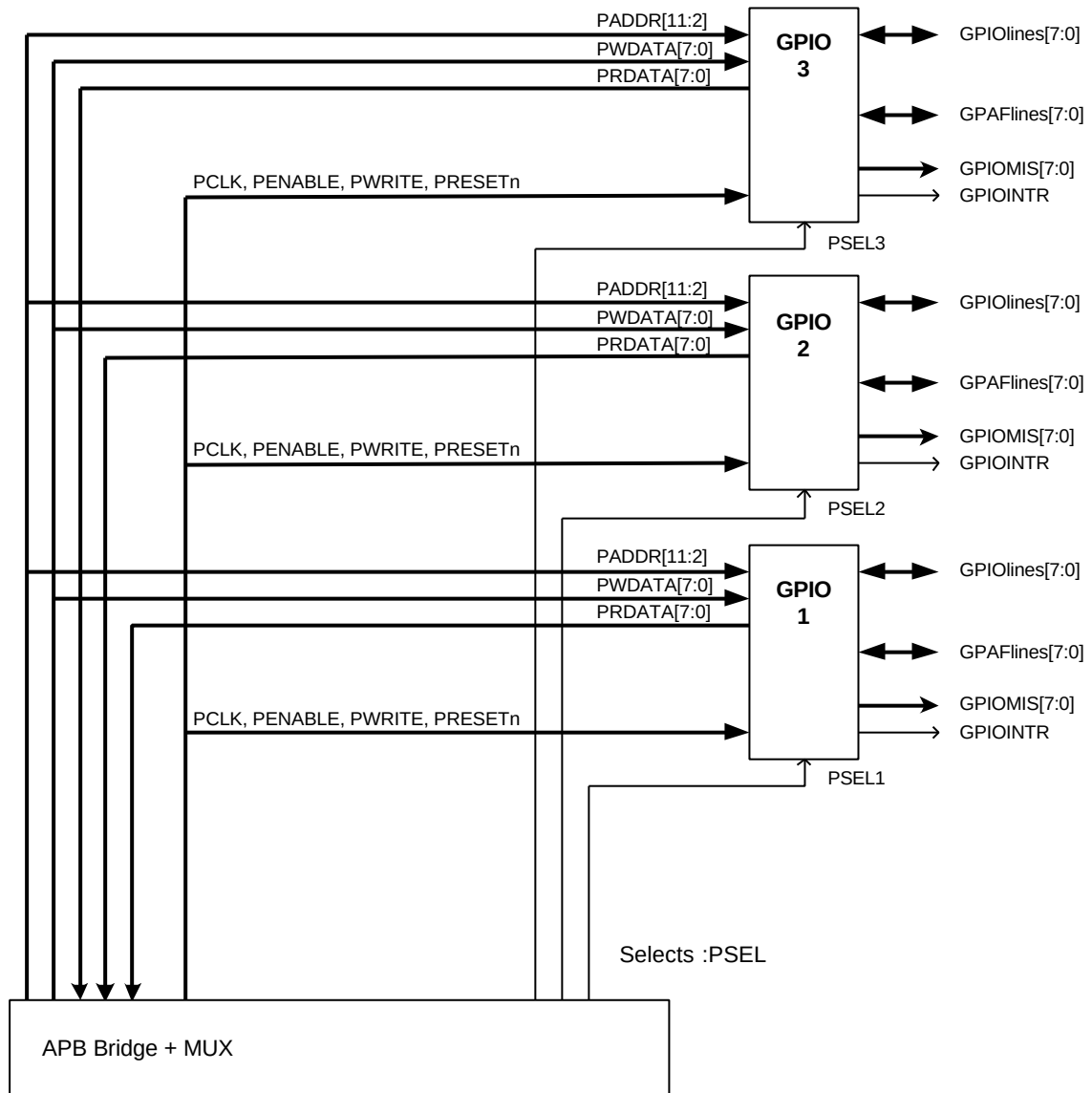


Figure 4.11-1. 24-bit GPIO created using three 8-bit PrimeCell GPIO.

Note that PCLK, PENABLE, PWRITE and PRESETn are common signals for all PrimeCell peripherals. The APB Bridge decodes the address and generates a peripheral select, PSELx for every GPIO instantiated. Only one select signal can be active during a transfer, [Reference 5].

4.12 Integration Testing

The GPIO integration test vector suite allows the integrator to verify that the GPIO has been connected into the target system correctly. These vectors test 3 classes of signals:

- a) AMBA signals (GPIO signals connected directly to the APB)
- b) Intra-Chip signals (GPIO signals connected to the Interrupt Controllers and to the generic peripheral/device connected to the hardware control lines)
- c) Primary I/O signals (GPIO signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The integration vectors are contained in the Gpio.c file in the integration/bustest directory. As delivered, these vectors are intended to run correctly with the GPIO in stand-alone mode i.e even before the GPIO has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the GPIO to APB (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the GPIO to hardware control signals and GPIO to Interrupt Controller (Intra-Chip signals) will **NEED** modification. The Gpio.c file has been embedded with NOTES listing the changes that the integrator would have to make in the test code.
- c) The integration vectors targeting the GPIO to Pad connectivity do **NOT** have to be modified by the integrator. However it is imperative that the integrator connect the Integration Trickbox correctly to these pads. [Please refer to the sample integration testbench – tbench.vhd[v] – in the integration/vhdl[verilog]/tbench directory]. It is also essential that during integration testing **ONLY** the Integration Trickbox is connected to these pads.

4.13 Production Testing

The GPIO is designed to be production tested using scan-insertion and ATPG.

4.13.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The GPIO design fully supports this scan test and ATPG methodology. As delivered the GPIO is synthesised with 1 scan-chain for the PCLK domain.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the GPIO block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running the automatically generated vectors (ATPG).

4.14 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in the design. Results tend to be optimistic since worst case timing paths may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the GPIO world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

eg : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable, which should be created as an empty variable.

eg, **setenv PERIPH <enter>**

The variable is then set to '**GPIO**' in the GPIO makefile, this enables the make utility to change the variable value for each peripheral in the design.

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

eg : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

eg : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

eg : **setenv HDL_SOURCE vhdl**
setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

eg : **setenv HDL_NETL vhdl**
setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

eg : **setenv TEST_ENV BUSTALK**
setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

eg : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

eg : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

eg : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

eg: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**